



COMP30026

Models of Computation

Cezary Kaliszyk and William Umboh

Lecture Lecture 11

Finite Automata



Reminder

Assignment 1

- Due end of the week

Consultation hours

- Make use of the student hub!
- Tuesday 11am-1pm



Last Lecture: Review

Resolution for Predicate Logic

With Unification and Factoring gives a *procedure*



Last Lecture: Review

Resolution for Predicate Logic

With Unification and Factoring gives a *procedure*

But we can decide some large fragments

Horn Formulas and Prolog

Practical use of predicate logic

Automated Reasoning. With equality!



Last Lecture: Review

Resolution for Predicate Logic

With Unification and Factoring gives a *procedure*

But we can decide some large fragments

Horn Formulas and Prolog

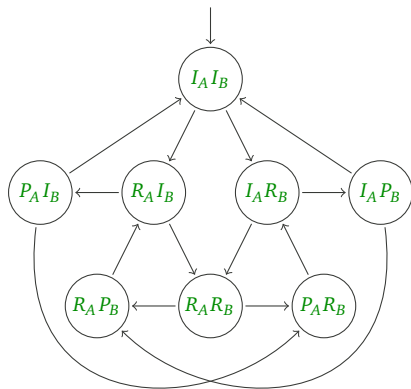
Practical use of predicate logic

Automated Reasoning. With equality!

First Part

Practically solve (very) hard problems

Reminder: Lecture 1



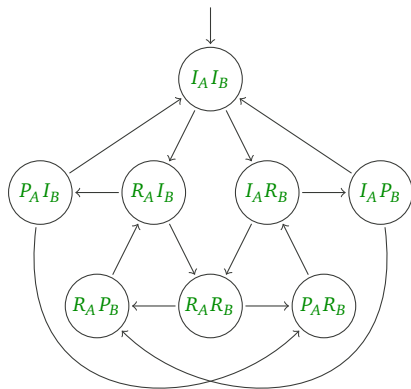
two users A and B

I_i user i is idle

R_i print request by user i

P_i printing document for user i

Reminder: Lecture 1



two users A and B

I_i user i is idle

R_i print request by user i

P_i printing document for user i

this is some program...
is there no comp-way...
to deal with it?



Computation

We want correct and efficient algorithms

- For every input x , what is the correct output?
- What are the resources needed (time, space, ...)
- Tradeoff between resources

We want correct and efficient algorithms

- For every input x , what is the correct output?
- What are the resources needed (time, space, ...)
- Tradeoff between resources

Impossibility results

- Can we show that something cannot be done faster?
- Can we show that something cannot be done?
- Impossibility results seem impossible...



Simplify, simplify, ...

Inputs

bit strings (Boolean strings)

Outputs

YES/NO

Computational problem

= set of YES inputs

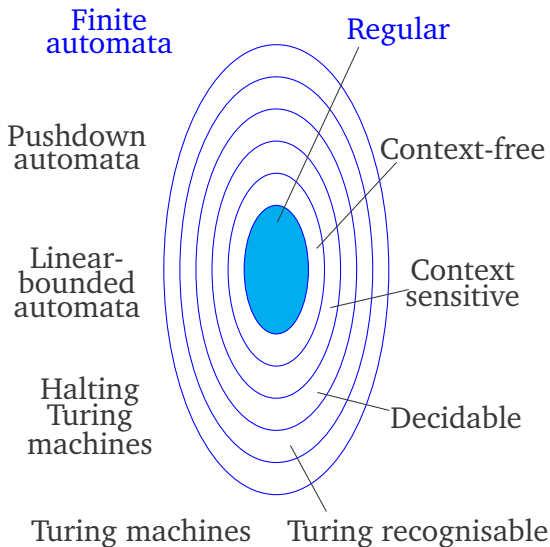
= language

First model

- No new memory
- Not even a variable

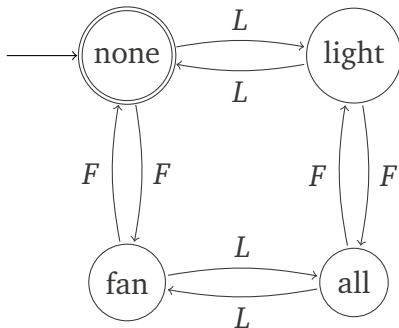


Machines vs Languages



An Example Automaton

A **state diagram** for simple controller for lighting and ventilation:

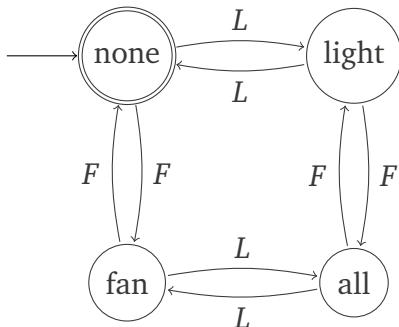


Start in the “none” state.

We can then toggle the light (*L*) or the fan (*F*).

An Example Automaton

A **state diagram** for simple controller for lighting and ventilation:

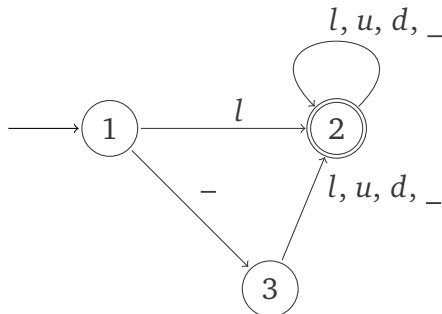


LFLFFF is accepted: it finishes in the “none” state.

LLFFL is rejected: it finishes in the “light” state.

Example 2

Recognise variable indentifiers in Haskell:



l is an abbreviation for $a, \dots, z$ (lower-case letters)

u is an abbreviation for $A, \dots, Z$ (upper-case letters)

d is an abbreviation for $0, \dots, 9$ (the digits)



Formal Definition

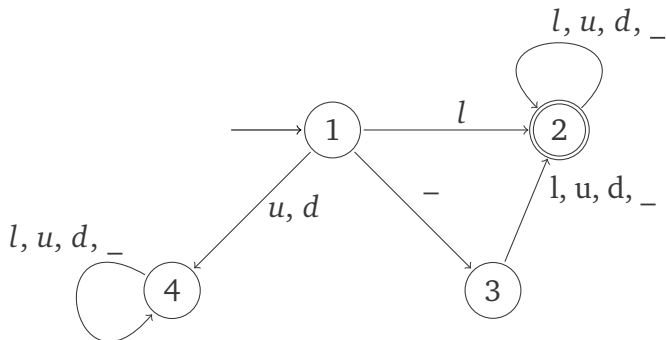
A **finite automaton** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

- Q is a finite set of **states**,
- Σ is a finite **alphabet**,
- $\delta : Q \times \Sigma \rightarrow Q$ is the **transition function**,
- $q_0 \in Q$ is the **start state**, and
- $F \subseteq Q$ are the **accept states**.

δ must be defined for all possible inputs.

Back to Example 2

To make it clear that the transition function is total, we should add a new state 4 and arcs to state 4 from state 1:





Strings and Languages

Alphabet, Σ

any non-empty finite set



Strings and Languages

Alphabet, Σ

any non-empty finite set

Symbols of the alphabet

elements of Σ . Usually a, b, c or 0, 1, 2, ...



Strings and Languages

Alphabet, Σ

any non-empty finite set

Symbols of the alphabet

elements of Σ . Usually a, b, c or 0, 1, 2, ...

String over Σ

A **finite** sequence of symbols from Σ



Strings and Languages

Alphabet, Σ

any non-empty finite set

Symbols of the alphabet

elements of Σ . Usually a, b, c or 0, 1, 2, ...

String over Σ

A **finite** sequence of symbols from Σ

Concatenation

concatenation of string x with a string y written as xy



Strings and Languages

Alphabet, Σ

any non-empty finite set

Symbols of the alphabet

elements of Σ . Usually a, b, c or 0, 1, 2, ...

String over Σ

A **finite** sequence of symbols from Σ

Concatenation

concatenation of string x with a string y written as xy

language (over alphabet Σ)

set of strings over Σ



Strings and Languages

Alphabet, Σ

any non-empty finite set

Symbols of the alphabet

elements of Σ . Usually a, b, c or 0, 1, 2, ...

String over Σ

A **finite** sequence of symbols from Σ

Concatenation

concatenation of string x with a string y written as xy

language (over alphabet Σ)

set of strings over Σ

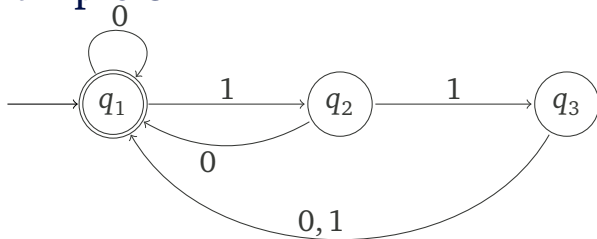
- empty string is denoted by ϵ
- **all strings** denoted Σ^*



Examples of Languages over Alphabet $\Sigma = \{0, 1\}$

- $\emptyset$
- $\{\epsilon\}$
- $\{\epsilon, 0, 1\}$
- $\{00, 01, 10, 11\}$
- $\{\epsilon, 0, 00, 000, \dots\}$
- $\{\epsilon, 0, 1, 00, 11, 000, 111, \dots\}$
- $\{\epsilon, 01, 0011, 000111, \dots\}$
- $\{w \in \Sigma^* \mid w \text{ contains odd number of } 0\}$
- $\{w \in \Sigma^* \mid \text{the length of } w \text{ is a multiple of } 3\}$
- $\{w \in \Sigma^* \mid w \text{ is not empty string}\}$
- $\{w \in \Sigma^* \mid w \text{ does not contain } 001\}$
- Σ^*

Example 3



The automaton M_1 (above) can be described precisely as

$$M_1 = (\{q_1, q_2, q_3\}, \{0, 1\}, \delta, q_1, \{q_1\}) \quad \text{with}$$

δ	0	1
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_1	q_1

$$L(M_1) = \left\{ w \in \Sigma^* \mid \begin{array}{l} w \text{ is } \epsilon, \text{ or ends with '0', or the number of} \\ \text{'1' symbols ending } w \text{ is a multiple of 3} \end{array} \right\}$$

is the language **recognised** by M_1 .

Acceptance, Formally

Let $M = (Q, \Sigma, \delta, q_0, F)$ and
let $w = v_1 v_2 \cdots v_n$ for some $v_i \in \Sigma$.

Definition

M **accepts** w iff there is a sequence of states $r_0, r_1, \dots, r_n$, with each $r_i \in Q$, such that

1. $r_0 = q_0$
2. $\delta(r_i, v_{i+1}) = r_{i+1}$ for $i = 0, \dots, n-1$
3. $r_n \in F$

Acceptance, Formally

Let $M = (Q, \Sigma, \delta, q_0, F)$ and
let $w = v_1 v_2 \cdots v_n$ for some $v_i \in \Sigma$.

Definition

M **accepts** w iff there is a sequence of states $r_0, r_1, \dots, r_n$, with each $r_i \in Q$, such that

1. $r_0 = q_0$
2. $\delta(r_i, v_{i+1}) = r_{i+1}$ for $i = 0, \dots, n-1$
3. $r_n \in F$

Let A be the set of all strings accepted by a machine M . We say A **is the language of** M and write $L(M) = A$.

We also say M recognises A .